

AYZE

XRPL Lending Protocol — Developer Feedback

XLS-65 Single Asset Vault · XLS-66 Lending Protocol · xrpl.js / TypeScript · lending-hackathon devnet

Context @0x7manny

We built the XRPL side of AYZE with xrpl.js in TypeScript: vault, liquidity, LoanBroker, first-loss cover, loan origination, repayments, default protection, lender withdrawal. The SDK was pleasant: one pattern covers almost everything.

```
client.request(...) · client.autofill(...) · wallet.sign(...) · client.submitAndWait(...)
```

Reads (ledger_entry, account_objects, vault_info) were straightforward too. The friction was not sending transactions; it was composing the primitives into one application, and understanding the parameters the spec fixes.

Top 3 improvements

1. An end-to-end XLS-65 + XLS-66 TypeScript example with state verification after each step.
2. Explicit Number → issued-currency precision guidance for repayments.
3. A troubleshooting table tied to real errors (tecPATH_DRY, tecTOO_SOON) and a short rationale for each spec cap.

1. Custom network onboarding @0x7manny

The workshop starts with `client.fundWallet()`; on the private hackathon endpoint it fails:

```
XRPLFaucetError: Faucet URL is not defined or inferrable.
```

Suggested improvement

- A hackathon-specific account bootstrap example at the top of the docs. First problem hit, about 15 minutes, unrelated to XLS-65/66.

2. Number precision during LoanPay @0x7manny

Ledger: PeriodicPayment 333.3358701198967648. Adding the service fee in JS gives 334.33587011989675; as an issued-currency Amount the SDK rejects it:

```
Decimal precision out of range
```

Both values look like ordinary decimal strings, so the different precision constraint is easy to miss. Cost: under 15 minutes once the error was read carefully, but it will hit every JavaScript integration.

Suggested improvement

- A warning in the LoanPay tutorial and a helper such as `toIssuedCurrencyAmount(value)` that rounds to the allowed precision.

3. LoanSet two-party signing @0x7manny

```
build LoanSet → autofill → LoanBroker signs → decode → signLoanSetByCounterparty() → submit
```

The helper `signLoanSetByCounterparty()` is excellent but easy to miss: we spent 30 to 60 minutes and 3 to 5 rejected submissions trying to add the counterparty signature with lower-level utilities before finding it. This was the single most expensive item. In AYZE both keys are custodial, so one server signs twice; with real wallets the half-signed blob has to travel to the borrower's device and no pattern is documented for that hand-off.

Suggested improvement

- Show the whole flow in one block, including why the fee covers the extra signature, and a note on exchanging the half-signed transaction between two parties.

4. TypeScript @0x7manny

xrpl.js 5.2 already ships typed VaultCreate, VaultDeposit, VaultWithdraw, LoanBrokerSet, LoanBrokerCoverDeposit, LoanSet, LoanPay, LoanManage, LoanDelete plus the flag enums: no raw JSON was needed for transactions. The only raw shape was the ledger_entry credential request (rippled wants credential_type, the type says credentialType).

```
import { Client, Wallet, type LoanBrokerSet } from "xrpl";
```

Suggested improvement

- Examples for the new transaction types that compile with tsc --noEmit, and aligned casing on the credential request.

5. Show state transitions, not only tesSUCCESS @0x7manny

The most useful information while building was what changed after a transaction. After a regular LoanPay:

```
PaymentRemaining 3 → 2 · DebtTotal 1000 → 666.667 · AssetsAvailable 4000 → 4333.335 ·
AssetsTotal 5000 → 5000.0037 · price/share 1 → 1.000000745
```

Liquidity is readable directly (vault_info.AssetsAvailable). Position and yield are not: price per share must be derived from AssetsTotal / shares.OutstandingAmount with Scale applied, the holder's shares come from account_objects type mptoken, utilisation needs DebtTotal from a second object, and STNumber fields can arrive as 1.5e9.

Suggested improvement

- A "Verify the resulting state" block after each tutorial step (VaultDeposit, VaultWithdraw, LoanBrokerCoverDeposit, LoanSet, LoanPay, default).
- Price per share and holder position returned by vault_info, or a documented recipe.

6. Error-oriented documentation @0x11semprez

Errors pointed at the symptom, not the cause: tecPATH_DRY was a missing TrustLine on the receiving account, nothing said so (about 15 minutes). tecTOO_SOON on LoanManage tfLoanDefault before due + GracePeriod was correct and, once known, became the guardrail our servicing loop relies on.

```
XRPLFaucetError → custom network, no faucet · tecPATH_DRY → missing TrustLine / path
Decimal precision out of range → issued-currency precision · tecTOO_SOON → default before
due + grace
```

Suggested improvement

- A troubleshooting table next to the flow with the first checks: tecPATH_DRY → issuer, destination TrustLine, path; tecTOO_SOON → NextPaymentDueDate, GracePeriod, ledger close_time.

7. Loan lifecycle after full repayment @0x11semprez

After tfLoanFullPayment the vault is liquid again, but PrincipalOutstanding, PaymentRemaining and NextPaymentDueDate change shape or disappear. Correct for the protocol; from the app it looked like a bug.

Suggested improvement

- A before / after example (ACTIVE → tfLoanFullPayment → COMPLETED) listing the fields an application can still rely on.

8. Parameters: unit, behaviour and rationale @0x7manny

Unit: CoverRateMinimum 90000 means 90 %, CoverRateLiquidation 66667 means 66.667 % (1/10 bps). Fine for the protocol, error-prone in application code; examples should use a percentToRate(90) helper.

Behaviour: the names suggest per-loan ratios, but `CoverRateMinimum` gates origination against the broker's total `DebtTotal`, and a default draws `min(DebtTotal × CoverRateMinimum × CoverRateLiquidation, DefaultAmount, CoverAvailable)`. Getting an exact 70 % first-loss per loan meant choosing 70 000 / 100 000 and checking cover per loan in the app. That took reading the reference implementation, not the docs.

Rationale: caps such as `ManagementFeeRate ≤ 10 000` (10 % of interest) or `InterestRate` annualised are stated but never explained, so an application that needs a different split (ours: 50/30/20 between protection seller, broker and lenders) has to guess whether to fight the cap or move the logic off-ledger. The docs live on GitHub and read as a spec; we would have liked several short examples of how XLS-65 and XLS-66 articulate, and one sentence of "why" per fixed percentage.

Suggested improvement

- Less verbose docs with worked examples of vault + broker + loan combinations, and the reasoning behind each cap.

9. Rates: make basis points the only unit @Keuchnotkush

Every rate in XLS-66 is an integer in 1/10 of a basis point (`CoverRateMinimum` 90000 = 90 %, `ManagementFeeRate` 2000 = 2 %), while application code and product discussions think in percent. Converting at the edge with floats is where rounding creeps in: 3 % becomes 0.03, times 100 000, and a value that should be exactly 3000 may serialise as 2999.9999 or be off by one after a chain of operations. The same thing bit us on amounts (section 2). The protocol representation is right; the problem is that nothing forces the caller to stay in integers.

Three ways to close the gap, from strongest to lightest:

- **Protocol / SDK type:** rate fields only accept an integer basis-point value, and the SDK refuses a float. `CoverRateMinimum`: 9000 (bps) with the $\times 10$ applied by the library.
- **Explicit helper in `xrpl.js`:** `bps(300)` for 3 %, `percent(3)` for the same, both returning the integer the ledger expects; raw numbers still allowed but examples never use them.
- **Decimal at the call site, conversion inside the SDK:** `ManagementFeeRate`: 0.03 and the library converts to 3000 at serialisation time, rounding once, with a documented rounding rule.

Any of the three keeps the maths in integers on the way to the ledger. We ended up writing our own bps helpers and keeping every amount in drops as bigint; a first-party version would stop each team from rediscovering the rule through a rejected transaction or a one-drop mismatch.

Suggested improvement

- Integer basis points as the single documented unit for rates, with a helper in the SDK and no float example anywhere in the Lending docs.

10. What worked well @0x7manny · @0x11semprez · @Keuchnotkush

- One pattern for every write: `submitAndWait(tx, { wallet, autofill: true })`; predictable reads.
- Primitives compose cleanly: Vault → LoanBroker → Loan → Repayment → Vault accounting.
- Observed behaviour matched the spec where it mattered: `tfLoanLatePayment / tecTOO_SOON` boundaries follow `NextPaymentDueDate + GracePeriod` exactly, which our servicing loop relies on.
- The custom explorer (`custom.xrpl.org` for the hackathon node) decoded Vault, LoanBroker and Loan objects and the new transaction types; we linked every hash to it from the dashboard and it never disagreed with the ledger reads.

Overall: the gap is composition

```
TrustLine → Vault → LoanBroker → Cover → Loan → Repayment → Vault accounting → Withdrawal
```

Each primitive is documented; a real application needs the chain. One complete TypeScript reference implementation, with preconditions, expected state before and after, common errors, types and value conversion rules, would have removed most of the friction on the XRPL side of AYZE.

The table of contents we wanted to find is the one we ended up writing ourselves:
docs/amendments/README.md in the AYZE repository maps every transaction of XLS-65, XLS-66 and XLS-70 to its signer, its fields, the function that submits it and the ledger reads that verify it.